Part 7 — Git, Monitoring & Security

Chapter 18: Git

Probability: 🔴 90%

Top Git Interview Questions

#	Question	Answer
1	`git merge` vs `git rebase`?	Merge creates a merge commit preserving history. Rebase replays commits on top of target — linear history. Use merge for shared branches, rebase for feature branches before merge
2	What is a fast-forward merge?	When target branch has no new commits since branch point. No merge commit needed. `git merge --ff`
3	How to resolve merge conflicts?	Edit conflicted files (remove markers), `git add`, `git commit`. Use `git mergetool` for GUI
4	What is `git stash`?	Temporarily saves uncommitted changes. `git stash` → do other work → `git stash pop`
5	What is `git cherry-pick`?	Apply a specific commit from another branch. `git cherry-pick <sha>`
6	How to undo last commit?	`git reset --soft HEAD~1` (keep changes staged). `git reset --hard HEAD~1` (discard changes)
7	What is `git reflog`?	Log of all HEAD movements. Recovery tool — find lost commits after reset
8	Explain `.gitignore`	File listing patterns to exclude from tracking. Prevents committing secrets, build artifacts, node_modules
9	What is a detached HEAD?	HEAD points to a commit, not a branch. Commits here are "orphaned" unless you create a branch
10	`git fetch` vs `git pull`?	Fetch: downloads remote changes but doesn't merge. Pull: fetch + merge (or rebase with `--rebase`)
11	What are Git hooks?	Scripts triggered by Git events. `pre-commit` (lint), `pre-push` (tests), `commit-msg` (format)
12	How to squash commits?	`git rebase -i HEAD~N` → change `pick` to `squash`. Clean commit history before merge
13	What is `git bisect`?	Binary search through commits to find which introduced a bug. `git bisect start`, `good`, `bad`
14	Branching strategy for your team?	Trunk-based: short-lived feature branches, merge to main via PR. Feature flags for incomplete work

#	Question	Answer
15	How to remove sensitive data from Git history?	`git filter-branch` or `BFG Repo-Cleaner`. Then force push. Rotate the exposed credential immediately

Branching Strategies

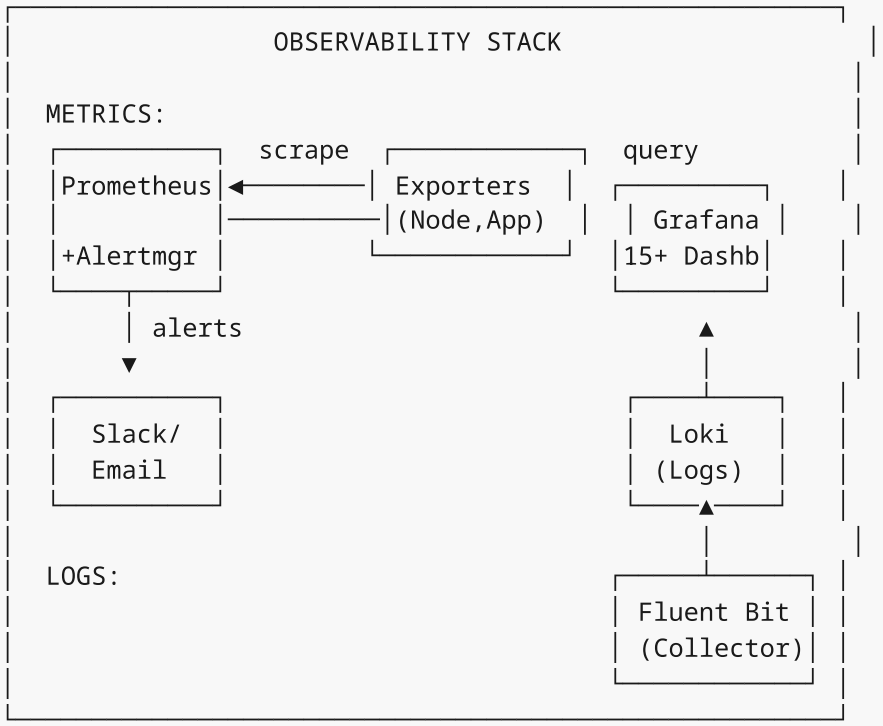
```
TRUNK-BASED (Recommended)
main ──●──●──●──●──●──●──●──●──●──
        \      /  \    /
feat-1   ●──●    feat-2 ●
(short-lived, 1-2 days)

GITFLOW (Complex teams)
main    ──●─────────────●─────●──
           \            / \     /
develop ──●──●──●──●──●   ●──●
             \  /
feature    ●──●
             release ──●──●
                       hotfix ──●
```

Chapter 19: Monitoring & Observability

Probability: 🟠 80%

Architecture (Your Setup)



Prometheus Concepts

Concept	Explanation
Pull-based	Prometheus SCRAPES targets (vs push-based like StatsD)
Metric types	Counter (always increases), Gauge (up/down), Histogram (distributions), Summary
PromQL	Query language. `rate(http_requests_total[5m])` = requests per second over 5 min
Service Discovery	Kubernetes SD, file-based, consul — automatically finds new targets
AlertManager	Receives alerts from Prometheus, deduplicates, groups, routes to Slack/email/PagerDuty
Recording Rules	Pre-compute expensive queries. Store results as new metrics

Key PromQL Examples

```
# Request rate per second
rate(http_requests_total[5m])

# 95th percentile latency
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Error rate percentage
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m])) * 100

# CPU usage per container
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)

# Memory usage percentage
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes)
/ node_memory_MemTotal_bytes * 100

# Disk space alert condition
(node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 10
```

Interview Questions

Q: "How do you set up alerting for a production service?"

> *"I define SLO-based alerts: 1) Availability: alert if error rate > 1% for 5 minutes. 2) Latency: alert if p99 > 500ms for 10 minutes. 3) Saturation: alert if CPU > 80% or memory > 85%. These fire to Alertmanager which groups related alerts, deduplicates, and routes to Slack (warning) or PagerDuty (critical). I avoid alert fatigue by using multi-window burn rates rather than simple threshold alerts."*

Q: "Prometheus vs Azure Monitor — how do they compare?"

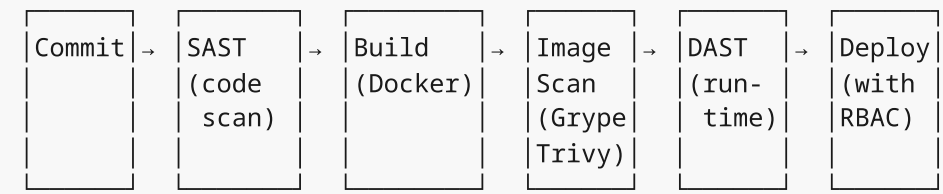
> *"Prometheus: open-source, pull-based, PromQL, runs in your cluster, you manage it. Azure Monitor: managed service, push-based (agents push to Azure), KQL query language, integrated with Azure*

> *services, auto-scales. My Prometheus+Grafana setup gives me the same capabilities. The advantage of managed monitoring (Azure Monitor) is zero operational overhead."*

Chapter 20: Security & DevSecOps

Probability: 🟡 70%

DevSecOps Pipeline



Key Security Topics

Topic	Your Experience	Interview Answer
Image Scanning	Grype in CI/CD pipeline	"We integrated Grype into our CI/CD security gate. Builds with critical/high CVEs are blocked. This reduced production vulnerability exposure by 90%."
RBAC	Kubernetes RBAC, Jenkins roles	"We use principle of least privilege. Kubernetes Roles scoped to namespaces, ClusterRoles for cluster-wide. Service accounts have minimal permissions."
Secrets	K8s Secrets, Jenkins credentials	"Never in code. K8s Secrets (encrypted at rest), Jenkins Credential Store, environment variables from CI/CD. For Azure: Key Vault with managed identity."
Network Security	NSGs, Security Groups, NetworkPolicy	"Defense in depth: cloud NSGs at network layer, K8s NetworkPolicies at pod layer, application-level auth."
Compliance	ISO 27001, SOC 2 support	"Supported ISO 27001 and SOC 2 compliance by enforcing access controls, centralized logging, backup validation, and change management workflows."
SSL/TLS	Certificate management	"TLS everywhere. cert-manager for automated Let's Encrypt certs in K8s. Internal mTLS for service-to-service."

Interview Questions

Q: "How do you handle a leaked secret in a Git repository?"

> *"1) Immediately rotate the credential. 2) Remove from Git history using BFG Repo-Cleaner. 3) Force push to all branches. 4) Audit access logs for the compromised credential. 5) Add pre-commit hooks to*

> *prevent future leaks (tools like git-secrets, detect-secrets). 6) RCA and team training. The credential rotation is the FIRST step — removing from Git history is important but the secret is already compromised."*

Q: "What is shift-left security?"

> *"Moving security earlier in the development lifecycle instead of only checking at deployment. Examples: SAST (static analysis) at commit time, dependency scanning in CI, container image scanning during build, policy-as-code for infrastructure. Our Grype integration is a shift-left example — we catch CVEs at build time, not in production."*

Q: "Explain RBAC in Kubernetes"

```
# Role — namespaced permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
# RoleBinding — binds Role to user/group/SA
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: production
  name: read-pods
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

> *"Role defines WHAT permissions. RoleBinding defines WHO gets them. Role is namespace-scoped. ClusterRole is cluster-wide. Always use least privilege — give only the permissions needed."*